

Sécurité Avancée des Bases de Données

Chiffrement – Secrets – Hardening – Audit



Objectifs du cours



Comprendre les mécanismes

Maîtriser les techniques essentielles pour protéger une base de données moderne contre les menaces actuelles



Suite logique RBAC/RLS

Approfondir la sécurité au-delà du contrôle d'accès pour une protection complète



Standards professionnels

Appliquer les bonnes pratiques de l'industrie pour sécuriser les données sensibles



Pourquoi chiffrer, durcir et auditer ?

Menaces réelles

- Attaques MITM
- Vol physique de disques
- Dumps illégaux
- Accès DBA malveillant

Obligations légales

Données sensibles → conformité RGPD obligatoire

Protection des informations personnelles identifiables

Les 3 piliers de la sécurité

Confidentialité

Garantir que seules les personnes autorisées accèdent aux données

Intégrité

Assurer que les données ne sont pas altérées ou corrompues

Traçabilité

Maintenir un historique complet des accès et modifications



Le chiffrement n'est pas une option – c'est un standard professionnel incontournable



Chiffrement en transit

Le problème des connexions non sécurisées

Sniffing réseau

Interception des identifiants et données en clair sur le réseau

Attaques MITM

Interposition entre application et base de données pour voler ou modifier les flux

Solution obligatoire

TLS sur toutes les connexions réseau – aucune exception

Comment fonctionne TLS ?



1. Client Hello

Le client initie la connexion sécurisée



2. Certificat serveur

Le serveur envoie son certificat SSL/TLS



3. Vérification

Validation de la clé publique du serveur



4. Canal chiffré

Établissement du tunnel sécurisé

Résultat : tout le trafic devient illisible pour les attaquants

TLS en pratique

Configuration par SGBD

PostgreSQL

```
ssl = on
```

Certificats serveur
et clients
obligatoires

MySQL

```
require_secure_  
transport = ON
```

Force toutes les
connexions en TLS

MongoDB

```
net.ssl.mode =  
requireSSL
```

Rejette les
connexions non
chiffrées



Bonne pratique : interdire systématiquement les
connexions non-TLS en production



Chiffrement au repos

Protection du stockage physique



Chiffrement disque

Protection des partitions, fichiers et volumes de stockage complets



Objectif principal

Sécuriser en cas de vol physique du serveur ou des disques durs



Limitation critique

Ne protège PAS contre un SELECT effectué par un DBA ou attaquant authentifié



TDE : Transparent Data Encryption

01

Chiffrement automatique

Fichiers data, journaux de transactions et backups

02

Support SGBD

MySQL ✓ | MongoDB ✓ |
PostgreSQL = solutions externes uniquement

03

Point critique

La clé maître (master key) nécessite une protection maximale

Bonnes pratiques

Chiffrement au repos

1

Chiffrer disque + backups

Aucune donnée ne doit être stockée en clair

2

Séparer les clés

HSM (Hardware Security Module) ou KMS (Key Management Service)

3

Rotation annuelle

Renouveler régulièrement les clés de chiffrement

4

Jamais dans Git

Ne jamais versionner les clés dans les dépôts de code



Chiffrement au champ

Protection granulaire des données

Pourquoi ?

- Protéger même si la DB est compromise
- Données PII, santé, messages privés
- Protection contre DBA malveillant
- Colonnes illisibles côté serveur

Cas d'usage

Identifiants sensibles, numéros de carte bancaire, informations médicales, conversations privées



Techniques de chiffrement au champ

Symétrique

Même clé pour chiffrer et déchiffrer

- ✓ Rapide et efficace
- ✓ Idéal pour volumes importants

Asymétrique

Paire clé publique/privée

- ✓ Plus sécurisé
- ⚡ Plus lent

Hash (unidirectionnel)

À sens unique – irréversible

- ✓ Parfait pour mots de passe
- ✓ bcrypt, Argon2

📌 **Important** : les champs chiffrés ne sont pas triables et difficilement indexables

PostgreSQL : pgcrypto

Fonctions principales

```
pgp_sym_encrypt(col, key)  
pgp_sym_decrypt(col, key)
```

Règle d'or : clés stockées hors
de la base de données

❏ **Limites** : impact sur les performances + impossibilité de
filtrer directement sur champ chiffré

Utilisation idéale

- Adresses email
- Numéros de téléphone
- Messages privés
- Identifiants sensibles





MongoDB : CSFLE

Client-Side Field Level Encryption



Chiffrement côté client

Les données sont chiffrées avant même d'être envoyées au serveur



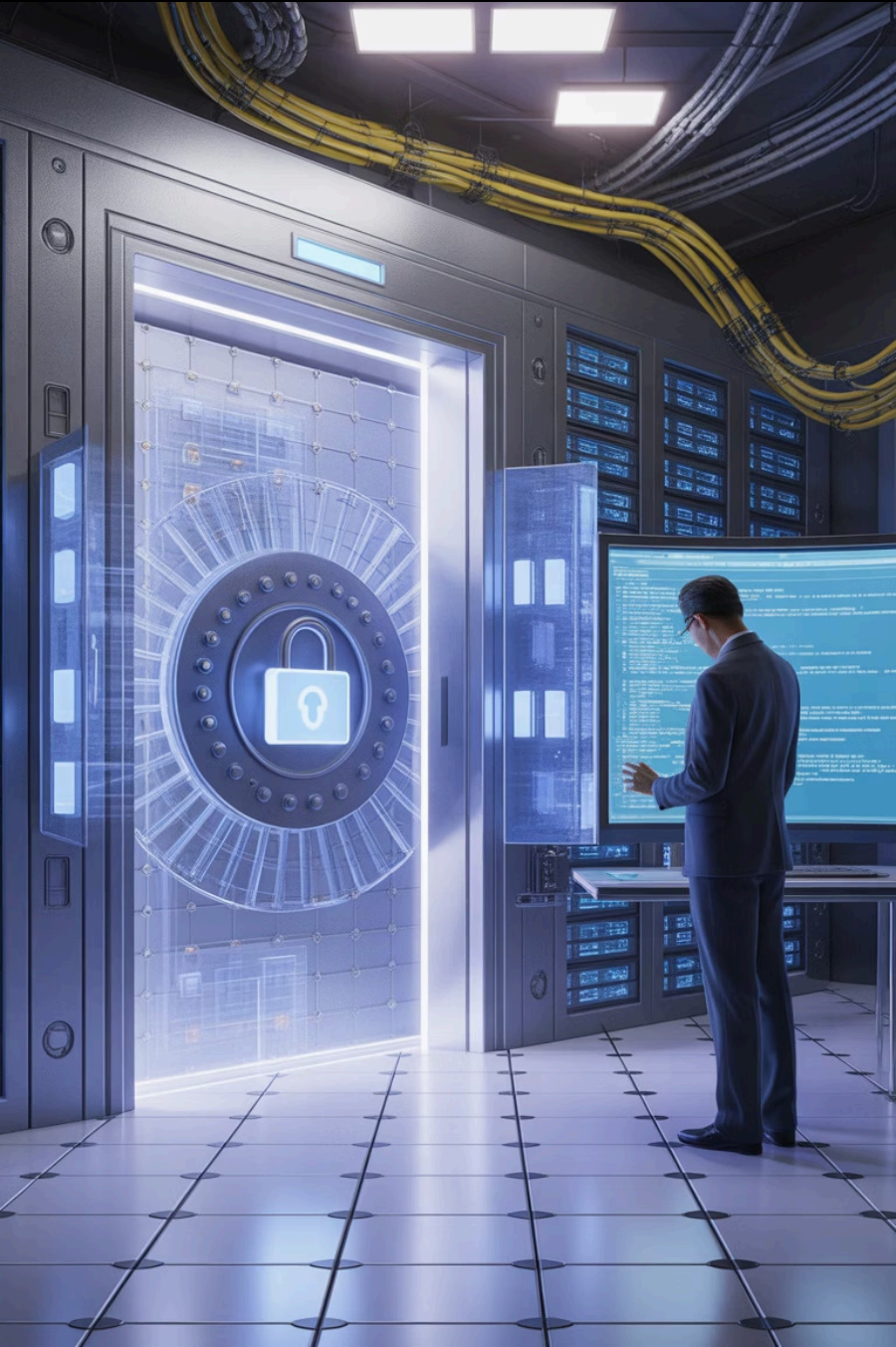
Stockage opaque

La base stocke uniquement du ciphertext illisible



Protection maximale

Même le DBA ne peut pas lire les données sensibles



Gestion des secrets

Le problème critique

Secrets = mots de passe DB, clés de chiffrement, tokens API, certificats

“

Erreurs fréquentes

- Clés versionnées dans Git
- Secrets en clair dans docker-compose
- Clés dans scripts ou logs applicatifs
- Permissions fichiers non restreintes

Conséquence → compromission instantanée du système

”

Solutions de gestion des secrets

Variables d'environnement

Fichiers .env avec permissions 600,
jamais dans Git

HashiCorp Vault

Gestion centralisée avec rotation
automatique et audit

Cloud KMS

AWS Secrets Manager, GCP KMS,
Azure Key Vault



Best practice : rotation automatique des secrets selon un calendrier défini

Durcissement (Hardening)

Objectifs stratégiques



Réduire la surface d'attaque

Minimiser les points d'entrée exploitables



Empêcher mauvais usages

Bloquer les pratiques dangereuses dès la configuration



RBAC minimal

Appliquer le principe du moindre privilège



Configuration sécurisée

Paramétrer le SGBD selon les standards de sécurité

Checklist de durcissement

1 Désactiver extensions inutiles
Réduire les fonctionnalités non essentielles

2 Désactiver login rôles applicatifs
Seuls les comptes de service doivent se connecter

3 Forcer SSL uniquement
Rejeter toute connexion non chiffrée

4 Restreindre listen_addresses
Limiter les interfaces d'écoute réseau

5 Séparer les rôles
Lecture / Écriture / Administration distincts

6 Désactiver fonctions dangereuses
COPY, file access, commandes système

Durcissement réseau & OS



Isolation réseau

- DB jamais exposée sur Internet
- Pare-feu restrictif (5432/3306/27017)
- Séparation stricte dev/test/prod

Protection stockage

- Sauvegardes chiffrées
- Stockage hors serveur
- Tests de restauration réguliers

Audit & Logging

Pourquoi auditer ?

Forensics

Comprendre ce qui s'est passé
après un incident



Conformité légale

Traçabilité obligatoire pour RGPD
et réglementations



Contrôle interne

Obligation pour systèmes critiques
et sensibles



Détection anomalies

Identifier les comportements
suspects en temps réel

